

软件学院本科生 2022—2023 学年第 1 学期计算机网络课程期末考试

试卷答案 (A 卷)

一、选择 (本题共 20 分, 每小题 2 分)

1 - 5 CDCDD

6 - 10 ABCDC

二、填空 (本题共 30 分, 每空 1 分)

1. 比特率、时延带宽积、带宽、吞吐量、包转发率、丢包率、时延、利用率、往返时间 RTT、时延抖动 (任选其二即可)
2. 电路交换、报文交换、分组交换
3. 传输时延/发送时延、传播时延、处理时延、排队时延
4. 快速以太网 千兆以太网
5. 转发/forwarding 过滤/filtering 泛洪/flooding
6. ICMP
7. 首部 数据
8. 距离向量路由
9. 复用 分用
10. <IP 地址, 端口号>二元组
 <源 IP 地址, 目的 IP 地址, 源端口号, 目的端口号>四元组
11. 半开连接
12. 应用层 互联网层/网际层 IP
13. 门限/ssthresh (两种写法均可)
14. 循环方式 并发方式
15. 权威域名服务器 递归解析器

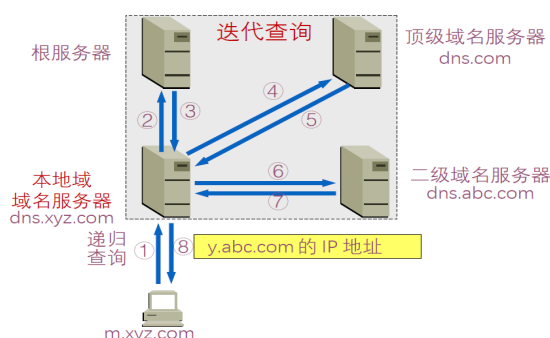
三、简答题 (本题共 10 分)

参考答案: 迭代查询方式流程, 具体见 PPT 应用层, 如下图。

域名解析过程

➤ 迭代查询

- 当收到查询请求报文的域名服务器不知道被查询域名的 IP 地址时, 就把自己知道的下一步应查询的域名服务器 IP 地址告诉本地域名服务器, 由本地域名服务器继续向该域名服务器查询, 直到得到所要解析的域名的 IP 地址, 或者查询不到所要解析的域名的 IP 地址
- 通常使用



四、计算题（本题共 15 分）

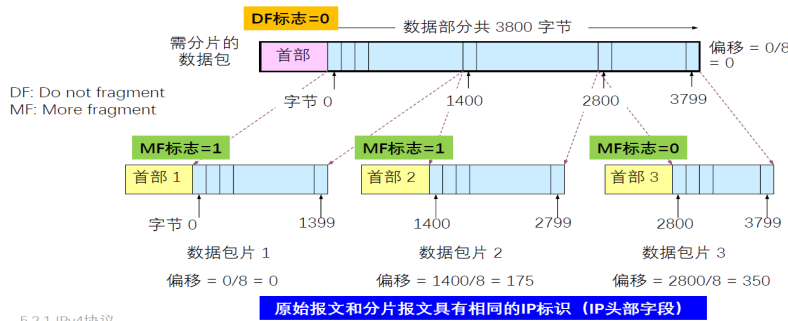
1. 数据包分片（7 分）

	总长度 (字节)	数据长度 (字节)	MF	DF	片偏移
原始数据报	3820	3800	0	0	0
数据报片 1	1420	1400	1	0	0/8=0
数据报片 2	1420	1400	1	0	1400/8=175
数据报片 3	1020	1000	0	0	2800/8=350

（具体可参考 PPT 网络层，如下图）



数据包分片



5.2.1 IPv4协议

原始报文和分片报文具有相同的IP标识（IP头部字段）

32

2. TCP 拥塞控制（8 分）

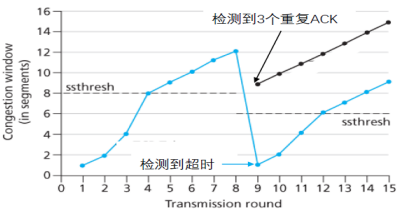
（1）关系曲线图可参考 PPT 传输层，如下图。（2 分）



TCP拥塞控制的实现



- 发送方维护变量 ssthresh
- 发生丢包时， $ssthresh = cwnd / 2$
- ssthresh 是从慢启动转为拥塞避免的分水岭：
 - cwnd 低于 ssthresh 时，执行慢启动
 - cwnd 高于 ssthresh 时：执行拥塞避免
- 拥塞避免阶段，拥塞窗口线性增长：
 - 每当收到 ACK， $cwnd = cwnd + MSS * (MSS / cwnd)$
 - 相当于每经过一个 RTT，cwnd 增加一个 MSS



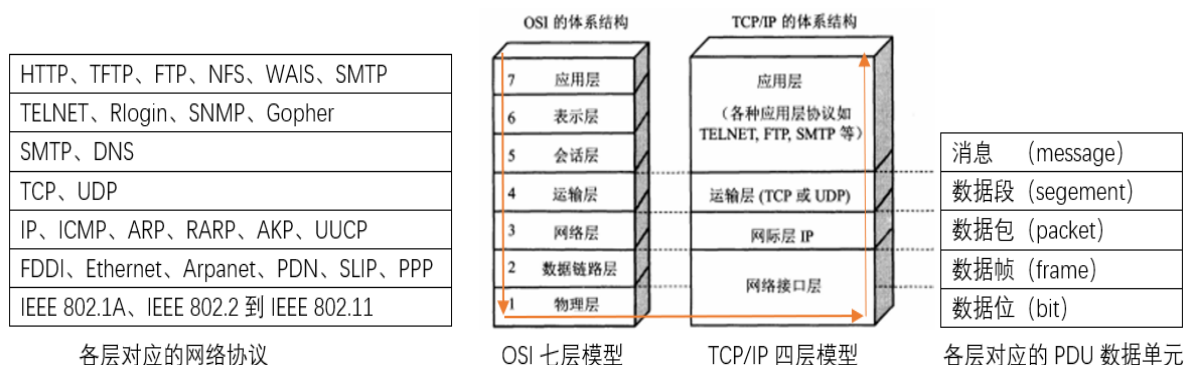
- 检测到3个重复的ACK后（黑色线）：
 - $cwnd = ssthresh + 3$ ，线性增长
- 检测到超时后（蓝色线）：
 - $cwnd = 1 \text{ MSS}$ ，慢启动

- (2) [1, 4] 和 [9, 12] （慢开始时间间隔看在哪一刻 2 倍增长停止）（1 分）
- (3) [4, 8] 和 [12, 15] （避免拥塞阶段看哪一阶段每一轮次只增加 1）（1 分）
- (4) ACK 曲线关系见上述 PPT 黑色线；（1 分）
关系式： $cwnd = ssthresh + 3$ （1 分）
- (5) 8 理由：在第 1 轮次发送时，门限 ssthresh 被设置为 cwnd 最大值的一半，即 $16 / 2 = 8$ 。（1 分）
- 6 理由：在第 10 轮次发送时，门限 ssthresh 被设置为上一轮拥塞窗口的一半， $12 / 2 = 6$ 。（1 分）

五、综合题（本题共 10 分）

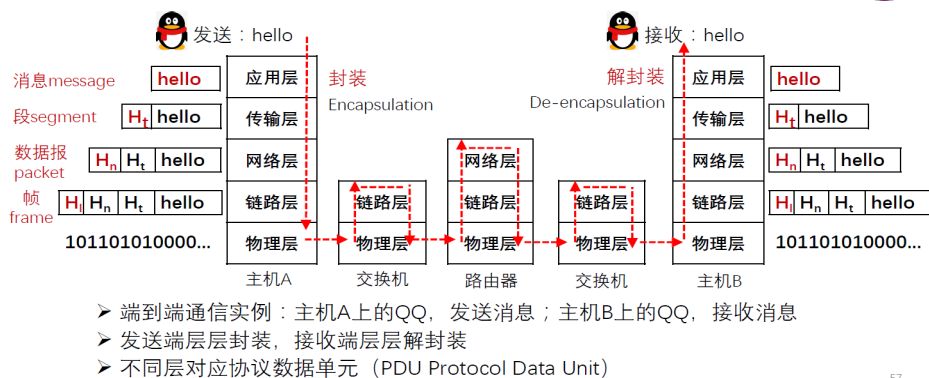
(1) 见下图。（3 分）

(2) 见下图。（4 分）

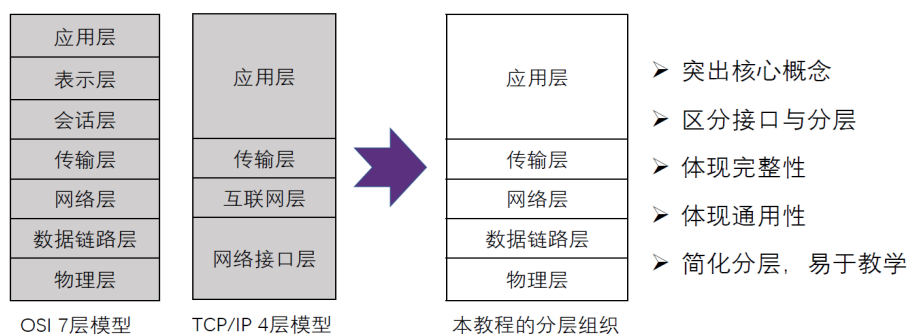


((1) (2) 具体可见 PPT 引言，如下两图的结合版。)

分层模型与网络实例



本教程内容的分层组织



(3) 具体可见 PPT 引言，如下两图的结合版。（3 分）



OSI模型与TCP/IP模型比较



➤ 7层模型与4层模型

- TCP/IP模型的网络接口层定义主机与传输线路之间的接口，描述了链路为无连接的互联网层必须提供的基本功能
- TCP/IP模型的互联网层、传输层与OSI模型的网络层、传输层大致对应
- TCP/IP模型的应用层包含了OSI模型的表示层与会话层

➤ 基本设计思想：通用性与实用性

- OSI：先有模型后设计协议，不局限于特定协议，明确了服务、协议、接口等概念，更具通用性
- TCP/IP模型：仅仅是对已有协议的描述

➤ 无连接与面向连接

- OSI模型网络层能够支持面向无连接和面向连接通信
- TCP/IP模型的网络层仅支持面向无连接通信（IP）

应用层
表示层
会话层
传输层
网络层
数据链路层
物理层

OSI 7层模型

应用层
传输层
互联网层
网络接口层

TCP/IP 4层模型



OSI模型与TCP/IP模型比较



OSI的失败：糟糕的时机、技术、实现、政策

OSI模型的不足

- 从未真正被实现
 - TCP/IP已成为事实标准，OSI缺少厂家支持
- 技术实现糟糕
 - OSI分层欠缺技术考虑：会话层、表示层很少内容；数据链路层、网络层内容繁杂。模型和协议过于复杂
 - 分层间功能重复：差错控制、流量控制等在不同层反复出现
- 非技术因素
 - TCP/IP实现为UNIX一部分，免费
 - OSI被认为是政府和机构的强加标准

TCP/IP模型的不足

- 核心概念未能体现
 - 未明确区分服务、接口和协议等核心概念
- 不具备通用性
 - 不适于描述TCP/IP之外的其它协议栈
- 混用接口与分层的设计
 - 链路层和物理层一起被定义为网络接口层，而非真正意思上的分层
- 模型欠缺完整性
 - 未包含物理层与数据链路层
 - 物理层与数据链路层是至关重要的部分

六、Socket 编程题 （15 分）

(1) 服务器端（8 分）

```
void server_without_conn()
{
    //加载socket库函数
    WORD wVersionRequested;
    WSADATA wsadata;
    int err;

    wVersionRequested = MAKEWORD(1, 1);
    err = WSASStartup(wVersionRequested, &wsadata);
    if (err != 0) return;
    if (LOBYTE(wsadata.wVersion) != 1 || HIBYTE(wsadata.wVersion) != 1) {
        WSACleanup();
        return;
    }

    //创建套接字
    SOCKET sockSrv = socket(AF_INET, SOCK_DGRAM, 0);
```

```

SOCKADDR_IN addSrv;

addSrv.sin_addr.S_un.S_addr = inet_addr("127.0.0.1");

addSrv.sin_family = AF_INET;
addSrv.sin_port = htons(2222);

int len = sizeof(SOCKADDR);
bind(sockSrv, (SOCKADDR*)&addSrv, len);

SOCKADDR_IN addrClient;
while (true) {
    printf("waiting for message...\n");
    char recvBuf[100];
    recvfrom(sockSrv, recvBuf, 100, 0, (SOCKADDR*)&addrClient, &len);
    char sendBuf[100];
    sprintf(sendBuf, "welcome %s", inet_ntoa(addrClient.sin_addr));
    sendto(sockSrv, sendBuf, strlen(sendBuf) + 1, 0, (SOCKADDR*)&addrClient, len);
    printf("%s\n", recvBuf);
}

closesocket(sockSrv);
WSACleanup();
return;

```

(2) 客户端 (7 分)

```

void client_without_conn()
{
    // 第一步, 加载socket库函数
    WORD wVersionRequested;
    WSADATA wsaData;
    int err;

    wVersionRequested = MAKEWORD(1, 1);

    err = WSStartup(wVersionRequested, &wsaData);
    if (err != 0) return;

    if (LOBYTE(wsaData.wVersion) != 1 || HIBYTE(wsaData.wVersion) != 1) {
        WSACleanup();
        return;
    }

    SOCKET sockClient = socket(AF_INET, SOCK_DGRAM, 0);

```

```
SOCKADDR_IN sockSrv;  
sockSrv.sin_addr.S_un.S_addr = inet_addr("127.0.0.1");  
sockSrv.sin_family = AF_INET;  
sockSrv.sin_port = htons(2222);  
  
char message[] = "Zihan Lin 2120210568";  
sendto(sockClient, message, strlen(message) + 1, 0, (SOCKADDR*)&sockSrv,  
sizeof(SOCKADDR));  
int len = sizeof(SOCKADDR);  
char recvBuf[100];  
recvfrom(sockClient, recvBuf, 100, 0, (SOCKADDR*)&sockSrv, &len);  
printf("%s\n", recvBuf);  
closesocket(sockClient);  
WSACleanup();  
return;  
}
```